Генератор случайных чисел MK61s mini

Версия документа: 08.09.2026

Назначение

У оригинального алгоритма К СЧ есть две проблемы для современного применения: после холодного старта калькулятор начинает с одного и того же внутреннего состояния, а измеренная последовательность быстро переходит в короткий цикл.

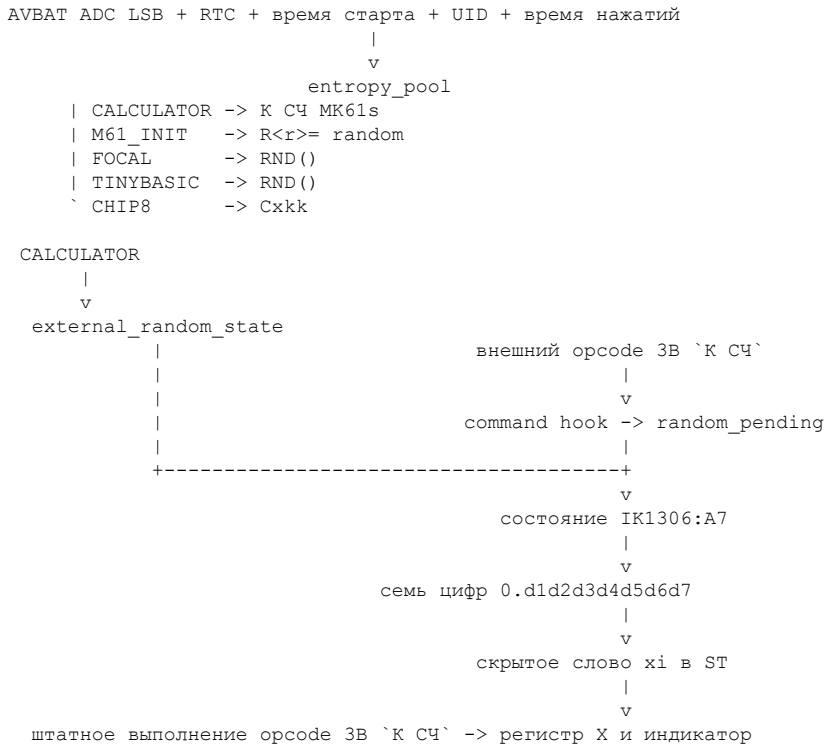
В MK61s mini предусмотрены два режима:

Настройка	Работа команды К СЧ
К СЧ MK61	Полностью штатный алгоритм и начальное состояние эмулируемого МК-61
К СЧ MK61s	Аппаратно инициализированный поток подставляет новое скрытое слово `xi`, после чего его обрабатывает штатная ROM-команда К СЧ

FOCAL, TinyBASIC, CHIP-8 и инициализатор регистров M61 используют собственные аппаратно инициализированные потоки. Настройка К СЧ на них не влияет.

Эта реализация предназначена для вычислений, моделирования и игр. Она не является криптографическим генератором и не должна применяться для ключей, паролей и других задач, где требуется криптографическая стойкость.

Общая схема



Здесь важно различать несколько уровней:

1. Пул смешивает физические измерения и временные события.
2. Из пула создаются независимые 64-битные состояния потоков.
3. Поток калькулятора инициализирует `external_random_state` ядра.
4. Command hook распознает внешний opcode 3B и разрешает одну подстановку.

5 В точке `IK1306:A7` из потока получается семизначное слово `xi`.

6 Итог в регистре X вычисляет штатный алгоритм МК-61. Он не равен напрямую выходу 64-битного генератора.

Сбор начального материала

Запуск во время заставки

`entropy_pool::begin()` вызывается до показа заставки. Сразу после него в пул добавляется первый снимок RTC. В каждом цикле ожидания заставки `entropy_pool::poll_startup()` выполняет одно измерение AVBAT. Поэтому время, которое раньше тратилось только на показ логотипа, одновременно используется для сбора начального материала.

После заставки вызывается `entropy_pool::finish_startup()`. Если пользователь пропустил заставку и данных еще мало, функция продолжает измерения до одного из двух условий:

- получено 64 бита после обработки фон Неймана;
- выполнено 1024 сырых измерения.

Затем восстанавливается обычная разрядность АЦП и создаются рабочие потоки. После этого в пул добавляется второй снимок RTC; он обновляет все потоки до их первого использования. Число 64 здесь означает целевое количество извлеченных битов AVBAT, а не доказанную оценку физической энтропии устройства.

Источник AVBAT

Для измерений используется 12-битный АЦП на входе AVBAT. Первое чтение после переключения внутреннего мультиплексора отбрасывается. Для каждого следующего чтения в пул целиком подмешиваются:

- сырое значение АЦП;
- значение `micros()`;
- номер измерения.

Для отдельного извлечения бита берется младший бит результата АЦП. Он может иметь смещение, поэтому пары младших битов обрабатываются методом фон Неймана:

```
00 -> отбросить
11 -> отбросить
01 -> 0
10 -> 1
```

Даже отброшенные пары не пропадают полностью: сырое значение и время чтения уже были подмешаны в состояние пула.

RTC: состояние часов и фаза субсекунд

RTC работает от независимого кварца LSE 32,768 кГц. При наличии основного питания backup-domain питается от VDD, а при его отключении может сохранять ход от VBAT. В пул дважды добавляется согласованный снимок, полученный последовательным чтением времени и даты STM32:

- календарные поля до секунды;
- признак, было ли время установлено пользователем;
- сырое значение нисходящего регистра `SubSeconds`;
- `SecondFraction`, задающий диапазон счетчика.

При штатных делителях F401/F411 синхронный счетчик имеет 256 положений за секунду: `SubSeconds` идет от 255 до 0 с шагом 3,90625 мс. Значение, которое библиотека называет миллисекундами, поэтому не содержит 1000 независимых состояний. Теоретический максимум состояния фазы равен 8 битам, но это не означает 8 битов случайности.

Календарное время предсказуемо и само по себе не является шумом. Фаза между независимым LSE и моментом запуска CPU может содержать джиттер, однако он пока не измерен отдельно. Поэтому RTC используется как материал для разнообразия и неповторяемости запусков, но ему консервативно приписывается 0 гарантированных битов энтропии. Счетчик `startup_entropy_bits()` учитывает только выход обработки фон Неймана для AVBAT.

Когда календарное время RTC не установлено, аппаратный счетчик идет, пока RTC получает питание, и его фаза может быть подмешана. Без батарейки после полного отключения backup-domain теряется: следующий холодный старт может дать тот же календарь и почти ту же фазу. Такой полностью повторяющийся снимок считается константой — он не добавляет энтропии, но и не ослабляет независимо собранный AVBAT или время нажатий. Если RTC не удалось прочитать, снимок пропускается. При установленном времени и подключенной батарее календарь дополнительно различает запуски, сделанные в разные моменты.

Дополнительные значения

При старте также подмешиваются `micros()`, `millis()` и, если платформа предоставляет `UID_BASE`, 96-битный идентификатор микроконтроллера. UID является постоянным и сам по себе не создает случайность, но разделяет состояния разных экземпляров устройства.

Смешивание использует 64-битную avalanche-функцию с финализатором SplitMix64. Она хорошо распространяет изменение входных битов по всему состоянию, но не превращает систему в криптографический генератор.

Проверка AVBAT с подключенной батареей

19.07.2026 на устройстве с CR2032 на VBAT записано 4096 последовательных 12-битных измерений с интервалом около 1 мс. Получены значения от 1011 до 1049; младший сырой бит имел 53,98% единиц и заметную корреляцию соседних значений около 0,36. Это подтверждает, что сырой LSB нельзя считать готовым равномерным случайным битом.

После обработки неперекрывающихся пар методом фон Неймана осталось 647 бит: 341 ноль и 306 единиц, корреляция соседних выходов около 0,07. В пределах этой записи для получения первых 64 выходных битов требовалось от 310 до 522 сырых измерений в зависимости от точки начала; каждое окно длиной 1024 измерения давало не менее 136 выходных битов.

Таким образом, подключенная батарея изменила статистику AVBAT и сделала сырые биты более коррелированными, но текущий предел 1024 измерения сохраняет заметный запас для целевых 64 выходов. Это достаточная инженерная проверка для игр, моделирования и вычислений, но не сертификация криптографической энтропии.

Независимые потоки

После завершения стартового сбора из общего пула создаются пять состояний с разными доменными константами:

Домен	Потребитель
`CALCULATOR`	Расширенный режим К СЧ MK61s
`FOCAL`	Функция `RND()` языка FOCAL
`TINYBASIC`	Функция `RND()` языка TinyBASIC
`CHIP8`	Инструкция случайного числа CHIP-8
`M61_INIT`	Правая часть `R<r>= random` в M61 и терминале

Чтение из одного потока не сдвигает остальные. Поэтому запуск программы FOCAL не меняет последовательность К СЧ, вызовы К СЧ не расходуют числа TinyBASIC, а `R<r>= random` не влияет ни на один из этих потоков.

Каждый поток развивается как SplitMix64: к состоянию прибавляется константа `0x9E3779B97F4A7C15`, затем применяется avalanche-функция. Интерфейс `next_u32()` сворачивает полученное 64-битное значение в 32 бита.

Учет времени нажатий

Каждое физическое нажатие клавиши добавляет в пул:

- код клавиши;
- отметку времени события в микросекундах;
- дополнительное текущее значение `micros()`.

Событие обновляет состояния всех доменов. Если включен К СЧ MK61s, для ядра калькулятора дополнительно формируется новый 64-битный материал и подмешивается в `external_random_state`.

Таким образом, начальное состояние появляется еще на заставке, а человеческие задержки между дальнейшими нажатиями продолжают изменять потоки. Нажатие обычной клавиши не показывает и не записывает случайное число в видимые регистры.

Выбор и хранение режима

Пункт находится в меню Настройки и показывает активное значение:

- К СЧ MK61;
- К СЧ MK61s.

Выбор хранится одним битом `random_mode_mk61s` в энергонезависимых настройках. При загрузке прошивки режим восстанавливается, а после запуска ядра вызывается `entropy_pool::configure_calculator()`.

Переключение в меню применяется сразу:

- для `MK61S` ядро получает новый 64-битный материал из домена `CALCULATOR`;
- для `MK61` оба встроенных hook отключаются.

Если переключить `MK61S` обратно в `MK61` без сброса, уже измененное скрытое слово `xi` остается внутри эмулируемой микросхемы, а дальше работает штатный алгоритм от этого состояния. После сброса с сохраненной настройкой `MK61` полностью восстанавливаются штатное начальное состояние и последовательность.

Использование двух hook API

К СЧ MK61s реализован как встроенный клиент двух общих API:

- command-hook API распознает внешний opcode `3B` из клавиатуры или программы;
- ROM-hook API дает точный доступ к ST в состоянии `IK1306:A7`.

Оба встроенных callback используют зарезервированные слоты. При выборе К СЧ MK61 они удаляются. По восемь публичных регистраций каждого API остаются доступны независимо от режима генератора.

Контракт внешних opcode описан в `MK61s-mini-Command-Hooks.md`, а правила доступа к внутренним R и ST - в `MK61s-mini-ROM-Hooks.md`.

Как число попадает в калькулятор

1. Инициализация состояния ядра

При включении режима `MK61S` пул выдает два 32-битных слова домена `CALCULATOR`. Они объединяются в 64-битный `seed_material`:

```
seed_material = (next_u32(CALCULATOR) << 32)
              |  next_u32(CALCULATOR)
```

Ядро смешивает его с отдельной константой и сохраняет результат в `external_random_state`. Это состояние скрыто от пользователя и не является регистром МК-61.

2. Обнаружение команды К СЧ

При включении К СЧ MK61s ядро регистрирует внутренний command callback для opcode `0x3B` в фазе `BEFORE_EXECUTE`.

Внешний opcode готов в IK1302 до начала штатного исполнения:

```
opcode = R[30] + 16 * R[33]
```

Для программы он перехватывается в точке декодера `IK1302:06`, для клавиатуры - в `IK1302:97`. Публичные callback исходной команды выполняются раньше встроенного распознавателя. Поэтому ГСЧ проверяет итоговый `replacement_opcode`:

```
external_random_enabled == true
replacement_opcode == 0x3B
```

При выполнении условий устанавливается одноразовый флаг `external_random_pending`. Замена `3B -> NOP` его не устанавливает, а замена другой команды на `3B` устанавливает.

Именно command hook доказывает связь с К СЧ. Сам адрес `A7` не является opcode и не распознает команду: разные внутренние пути могут посещать одинаковые адреса ROM, а содержимое ST имеет смысл только в конкретный момент.

3. Получение семи десятичных цифр

Отдельный ROM callback зарегистрирован для `IK1306:A7`. Он работает только при установленном `external_random_pending`. В этот момент состояние сдвигается еще на один шаг SplitMix64, флаг сразу снимается, и из результата вычисляется целое число:

```
seed7 = 1 + avalanche(external_random_state) mod 9999999
```

Диапазон `seed7` равен от 1 до 9 999 999. Число дополняется ведущими нулями до семи цифр и трактуется как логическое значение:

```
0.d1d2d3d4d5d6d7
```

Нулевое слово `0.0000000` намеренно исключено.

Если IK1306 посещает `A7` без ранее распознанного opcode `3B`, callback ничего не записывает и не сдвигает поток.

4. Запись в скрытое слово `xi`

ST IK1306 - транзитный рабочий массив, а не постоянный регистр `xi`. Только в единственный проверенный момент перед `A7` слово `xi` выровнено по приведенным ниже позициям. Callback срабатывает после определения адреса, но до декодирования ROM-команды, и записывает цифры в обратном порядке, начиная с младшей:

Индекс ST	Значение
1	`0`
4	`d7`
7	`d6`
10	`d5`
13	`d4`
16	`d3`
19	`d2`
22	`d1`
25	`0`
28	`9`
31	`9`

Индекс ST	Значение
34	9
37	0
40	0

Три служебные тетрады 999 входят в корректный формат внутреннего слова xi. Такая раскладка дает допустимое BCD-число и не создает ЕГГ0Г или сверхчисло.

Запись выполняется через context.st только в скрытый ST микросхемы IK1306. Callback не меняет регистры X, Y, Z, T, память, стек пользователя или индикатор.

Вне точки A7 эта раскладка неверна. Следующий микрошаг сразу потребляет данные; seed не хранится в ST после старта и не записывается из произвольного места основного цикла.

5. Завершение штатной ROM-командой

Сразу после подстановки эмулятор продолжает обычный микрошаг. Командное слово ROM в точке A7 читает новое xi, продолжает алгоритм пользовательской команды 0x3B и штатным путем помещает результат в X и на индикатор.

Поэтому пользователь видит именно результат алгоритма МК-61, запущенного от нового скрытого состояния. Прошивка не записывает готовое псевдослучайное число в X и не подменяет поведение остальных команд.

Почему подстановка выполняется при каждом К СЧ

Одной подстановки после старта недостаточно. В измеренной последовательности штатного алгоритма для проверенного начального состояния обнаружены 26 значений до входа в цикл и цикл длиной 153 значения, всего 179 различных значений до повтора.

Если дать внешний xi только первому вызову, дальнейшая последовательность снова будет развиваться только штатным алгоритмом и может попасть в короткий цикл. Поэтому в режиме MK61s свежее семизначное xi подставляется перед каждым выполнением К СЧ, а ROM каждый раз преобразует его обычным способом.

Почему ROM не патчится

Таблицы ROM остаются без изменений. Внешний command callback временно отмечает исполнение 3B, а ROM callback использует точно определенное внутреннее состояние IK1306:A7.

Это сохраняет несколько полезных свойств:

- режим MK61 может полностью отключить расширение;
- вся арифметика и видимый эффект команды остаются штатными;
- нет необходимости размещать новый генератор в микрокоде исторической ROM;
- изменение изолировано от остальных команд эмулятора;
- command-hook API можно использовать для подмены других внешних opcode;
- ROM-hook API остается доступен для независимых низкоуровневых расширений.

Фактически расширяется источник скрытого состояния xi, а не заменяется команда К СЧ.

FOCAL и TinyBASIC

FOCAL и TinyBASIC не вызывают К СЧ эмулируемого калькулятора. Их RND() берет старшие 24 бита очередного 32-битного значения своего домена и делит их на 16777216.0:

```
RND = (next_u32(domain) >> 8) / 16777216.0
```

Результат находится в диапазоне [0, 1). Оба языка всегда используют аппаратно инициализированные независимые потоки, даже когда в настройках выбран К СЧ МК61.

При сборке host-тестов вместо аппаратного пула используются фиксированные тестовые генераторы. Это сделано для воспроизводимости автоматических тестов и не относится к прошивке устройства.

Инициализация регистра M61

Правая часть `R<r>= random` берёт значение только из домена `M61_INIT` и равномерно преобразует его в одно из 9 999 999 ненулевых семизначных слов. Отклонение неполного остатка диапазона `uint32_t` устраняет modulo bias. Число записывается в тетрады регистра напрямую как `0.0000001`…`0.9999999`, без преобразования через `double` и без выполнения микропрограммы К СЧ.

Поэтому команда доступна и при выбранном режиме К СЧ MK61, не меняет скрытое слово `xi` и не сдвигает поток `CALCULATOR`. Как и весь пул, этот интерфейс предназначен для игр и моделирования, а не для криптографических задач.

Сохранение состояния при внутренних вычислениях

При математическом backend `CORE` FOCAL может временно использовать ядро МК-61 для вычисления функций. Снимок контекста включает не только регистры и память эмулятора, но также `external_random_enabled`, `external_random_pending` и `external_random_state`.

После внутреннего вычисления контекст восстанавливается. Поэтому служебный вызов математической функции не сдвигает пользовательскую последовательность К СЧ MK61s.

Проверки

Автоматические проверки подтверждают следующие свойства:

- несколько внешних opcode и внутренних ROM-адресов могут быть зарегистрированы одновременно;
- callback одного opcode выполняются в порядке регистрации;
- `D7 -> D8` работает из обоих источников, а `AFTER(D7)` меняет X без повреждения памяти;
- удаление независимо, оба реестра сохраняют восемь публичных слотов при ГСЧ;
- `3B -> NOP` не сдвигает поток, а `D7 -> 3B` использует усиленный seed;
- в `MK61` внешние seed не меняют первый результат, а в `MK61S` меняют;
- результат `MK61S` остается в диапазоне `[0, 1)`;
- обычная клавиша не раскрывает скрытый поток;
- математический backend не сдвигает поток, а выборка из 256 значений не наследует короткий штатный цикл.

На устройстве штатный режим повторял первое значение `0.4040670`, а `MK61S` после двух перезапусков дал `0.3381330` и `0.9296102`. В выборке из 10 000 вызовов `MK61S` получено 9957 разных результатов со средним около `0.4949`. Это инженерная проверка, а не доказательство криптографического качества.

Режим совместимости MK61

Когда выбран К СЧ MK61, `external_random_enabled` имеет значение `false`, а оба внутренних hook удалены. ROM, скрытое слово `xi` и все последующие переходы обрабатываются так же, как в исходном эмуляторе.

Таким образом, после холодного старта этот режим нужен для программ, тестов и демонстраций, которым важна исторически воспроизводимая последовательность. Режим `MK61S` предназначен для случаев, где повтор одного и того же старта и короткий цикл нежелательны.

Связанные файлы исходного кода

Основная реализация находится в `code/entropy_pool.{hpp,cpp}` и `code/mk61emu_core.cpp`; RTC-материал читается в `code/rtc_clock.{hpp,cpp}`; стартовый сбор и настройки - в `code/mk61s-M.ino`, `code/startup_splash.cpp`, `code/keyboard.cpp`, `code/menu.cpp`, `code/tools.hpp`; языковые `RND()` - в `code/focal.cpp`, `code/tinybasic.cpp`; проверки - в `tests/mk_math_self_test.cpp`.